

14.13. Exploring Counting Sort

- ♦ Counting Sort is not a Comparison based Algorithm
- ♦ Counting Sort counts the occurrences of elements instead of comparing the elements.
- ♦ Counting Sort is stable sorting Algorithm
- ♦ Counting Sort works well when you have range of Integer Values
- ♦ Counting sort assumes that each of the elements is an integer in the range 1 to K, for some integer K.

Limitations or When I can not use Counting Sort:

- ♦ When Array contains Negative Values
- ♦ When Array contains Floating point Values
- ♦ When Array contains Objects like Students, Customers

Complexity of Counting Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Average	$O(n)$
Time Complexity - Worst	$O(n)$
Space Complexity	$O(k)$

14.13.1. Counting Sort Implementation

Lab25.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
```



```
public class Lab25 {
    public static void countingSort(int arr[],int k) {

        int n = arr.length;
        int count[] = new int[k];

        for (int i = 0; i < n; i++) {
            count[arr[i]]++;
        }

        for (int i = 1; i < k; i++) {
            count[i] = count[i] + count[i-1];
        }

        int output[] = new int[n];
        for (int i = n-1; i >= 0; i--) {
            output[count[arr[i]]-1] = arr[i];
            count[arr[i]]--;
        }

        for (int i = 0; i < n; i++) {
            arr[i] = output[i];
        }
    }

    public static void main(String[] args) {

        int arr[] = { 2,5,5,2,0,1,2};

        System.out.println(Arrays.toString(arr));
        countingSort(arr,6);
        System.out.println(Arrays.toString(arr));

    }
}
```



14.14. Exploring Radix Sort

- ♦ Use Radix Sort when you have large Range Values
- ♦ Radix Sort is not a Comparison based Algorithm,
- ♦ It counts the occurrences of elements instead of comparing the elements
- ♦ Radix Sort is stable sorting Algorithm
- ♦ Radix Sort uses Counting Sort as Sub-Function

Algorithm:

1. Take the least significant digit of each element.
2. Sort the list of elements based on that digit, but keep the order of elements with the same digit.
3. Repeat the sort with each more significant digit.

Complexity of Cycle Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Average	$O(n)$
Time Complexity - Worst	$O(n)$
Space Complexity	$O(d)$

14.14.1. Radix Sort Implementation

Lab26.java

```
package com.jlcindia.sorting;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab26 {
    public static void radixSort(int arr[]) {

        int n = arr.length;
        int max= arr[0];
```



```
        for(int i=1;i<n;i++) {
            if(arr[i]>max) {
                max= arr[i];
            }
        }
        for(int exp =1;max/exp>0;exp = exp*10) {
            countingSort(arr,exp);
        }
    }
    public static void countingSort(int arr[],int exp) {

        int n = arr.length;
        int count[] = new int[10];

        for (int i = 0; i < n; i++) {
            count[(arr[i]/exp)%10]++;
        }

        for (int i = 1; i < 10; i++) {
            count[i] = count[i] + count[i-1];
        }

        int output[]= new int[n];

        for (int i = n-1; i>=0; i--) {
            output[count[(arr[i]/exp)%10]-1] = arr[i];
            count[(arr[i]/exp)%10]--;
        }

        for (int i = 0; i < n; i++) {
            arr[i] = output[i];
        }
    }
}
```

```
public static void main(String[] args) {  
  
    int arr[] = { 319,212,6,8,100,50};  
  
    System.out.println(Arrays.toString(arr));  
    radixSort(arr);  
    System.out.println(Arrays.toString(arr));  
  
}  
}
```

14.15. Exploring Bucket Sort

- ◆ When Values are uniformly distributed then use Bucket Sort.
- ◆ Like Counting sort, Bucket sort also imposes restrictions on the input to improve the performance.
- ◆ In other words, Bucket sort works well if the input is drawn from fixed set.

Ex:

Assume that all the input elements from $\{0, 1, \dots, K - 1\}$,
i.e., the set of integers in the interval $[0, K - 1]$.

That means, K is the number of distant elements in the input.

Complexity of Bucket Sort:

Time Complexity - Best	$O(n)$
Time Complexity - Average	$O(n)$
Time Complexity - Worst	$O(n^2)$
Space Complexity	$O(n)$



14.15.1. Bucket Sort Implementation

Lab27.java

```
package com.jlcindia.sorting;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab27 {
    public static void bucketSort(int arr[], int k) {
        int n = arr.length;

        //1. Find max Value
        int max_value=arr[0];
        for(int i=1;i<n;i++) {
            max_value=Math.max(max_value, arr[i]);
        }
        max_value++;

        //2. Create Buckets
        ArrayList<ArrayList<Integer>> mybuckets =
            new ArrayList<ArrayList<Integer>>();

        for(int i=0;i<k;i++) {
            mybuckets.add(new ArrayList<Integer>());
        }

        //3. Fill Buckets
        for(int i=0;i<n;i++) {
            int bucketIndex=(k*arr[i])/max_value;
            mybuckets.get(bucketIndex).add(arr[i]);
        }
    }
}
```



```
//4. Sort Buckets
for(int i=0;i<k;i++) {
    Collections.sort(mybuckets.get(i));
}

//5. Join Buckets
int myindex=0;
for(int i=0;i<k;i++) {
    for(int j=0;j<mybuckets.get(i).size();j++) {
        arr[myindex++]=mybuckets.get(i).get(j);
    }
}

public static void main(String[] args) {
    int arr[]={30,40,10,80,5,12,70,45};

    System.out.println(Arrays.toString(arr));
    bucketSort(arr, 4);
    System.out.println(Arrays.toString(arr));
}
}
```